

Spring Framework



비밀번호 찾기

비밀번호 찾기

요구 사항

- 아이디와 이메일이 DB의 회원과 동시에 일치하면 재설정 화면 이동, 불일치 복귀
- 공통 validation.js를 생성하여 빈칸 제어 적용
- 비밀번호 일치 확인도 별도의 js(reset_password.js) 생성
- register_member.jsp만 해당 js적용

비밀번호 찾기

login_member.jsp 수정

```
<p class="text-center mt-4 mb-0">
    <a href="registermember" class="text-decoration-none fw-semibold">회원가입</a>
    <a href="findpassword" class="text-decoration-none fw-semibold ms-3">비밀번호 찾기</a>
</p>
```

비밀번호 찾기

find_password.jsp 수정

```
<div class="card shadow rounded-4 mx-auto" style="max-width: 450px;">
    <div class="card-body p-4">
        <h4 class="card-title text-center fw-bold mb-4">비밀번호 찾기</h4>
        <form method="post" action="findpasswordconfirm" data-validate>
            <label class="form-label">아이디</label>
            <input type="text" name="m_id" class="form-control mb-3" placeholder="아이디를 입력하세요"
            data-label="아이디">

            <label class="form-label">이메일</label>
            <input type="text" name="m_mail" class="form-control mb-4" placeholder="이메일 주소를 입력하세요"
            data-label="이메일">

            <div class="d-grid">
                <button type="submit" class="btn btn-success btn-lg fw-bold">비밀번호 찾기</button>
            </div>
        </form>

        <p class="text-center mt-4 mb-0">
            <a href="loginmember" class="text-decoration-none fw-semibold">로그인</a>
            <a href="registermember" class="text-decoration-none fw-semibold ms-3">회원가입</a>
        </p>
    </div>
</div>
<script src="/js/validation.js"></script>
```

비밀번호 찾기

MemberDao.java 수정

- getMemberByMail() 추가

```
public MemberVo getMemberByIdMail(String m_id, String m_mail) {
    String sql = "SELECT * FROM member WHERE m_id = ? AND m_mail = ?";
    try {
        return jdbcTemplate.queryForObject(sql, new MemberRowMapper(), m_id, m_mail);
    } catch (EmptyResultDataAccessException e) {
        return null;
    }
}
```

비밀번호 찾기

MemberController.java 수정

- findpassword(), findpasswordconfirm() 추가

```
@RequestMapping("/findpassword")
public String findpassword() {
    return "user/find_password";
}

@RequestMapping("/findpasswordconfirm") //회원정보로 초기화
public String findpasswordconfirm(@RequestParam("m_id") String m_id,
        @RequestParam("m_mail") String m_mail, HttpSession session) {
    MemberVo member = memberDao.getMemberByIdMail(m_id, m_mail);
    if (member == null) {
        return "redirect:/findpassword";
    }
    session.setAttribute("resetMember", member);
    return "redirect:/resetpassword";
}
```

비밀번호 찾기

reset_password.jsp 추가

- src/main/webapp/WEB-INF/views/user/에 추가

```
<div class="card shadow rounded-4 mx-auto" style="max-width: 450px;">
    <div class="card-body p-4">
        <h4 class="card-title text-center fw-bold mb-4">비밀번호 재설정</h4>

        <form method="post" action="resetpasswordconfirm" data-validate>
            <label class="form-label">새 비밀번호</label>
            <input type="password" name="m_pw" id="m_pw" class="form-control mb-3" placeholder="새 비밀번호"
                data-label="새 비밀번호">

            <label class="form-label">새 비밀번호 확인</label>
            <input type="password" id="m_pw2" class="form-control mb-4" placeholder="한 번 더 입력"
                data-label="새 비밀번호 확인">

            <div class="d-grid">
                <button type="submit" class="btn btn-success btn-lg fw-bold">변경하기</button>
            </div>
        </form>
    </div>
</div>
<script src="/js/validation.js"></script>
<script src="/js/reset_password.js"></script>
```

비밀번호 찾기

reset_password.js 추가

- src/main/resources/static/js/에 추가

```
document.addEventListener("DOMContentLoaded", function () {

    document.querySelector("form").addEventListener("submit", function (e) {
        if (e.defaultPrevented) return;
        const pw = document.getElementById("m_pw").value;
        const pw2 = document.getElementById("m_pw2").value;
        if (pw !== pw2) {
            e.preventDefault();
            alert("새 비밀번호가 서로 다릅니다.");
        }
    });
});
```

비밀번호 찾기

MemberDao.java 수정

- updatePassword() 추가

```
public void updatePassword(int m_no, String m_pw) {
    String sql = "UPDATE member SET m_pw = ?, m_mod_date = now() WHERE m_no = ?";
    jdbcTemplate.update(sql, m_pw, m_no);
}
```

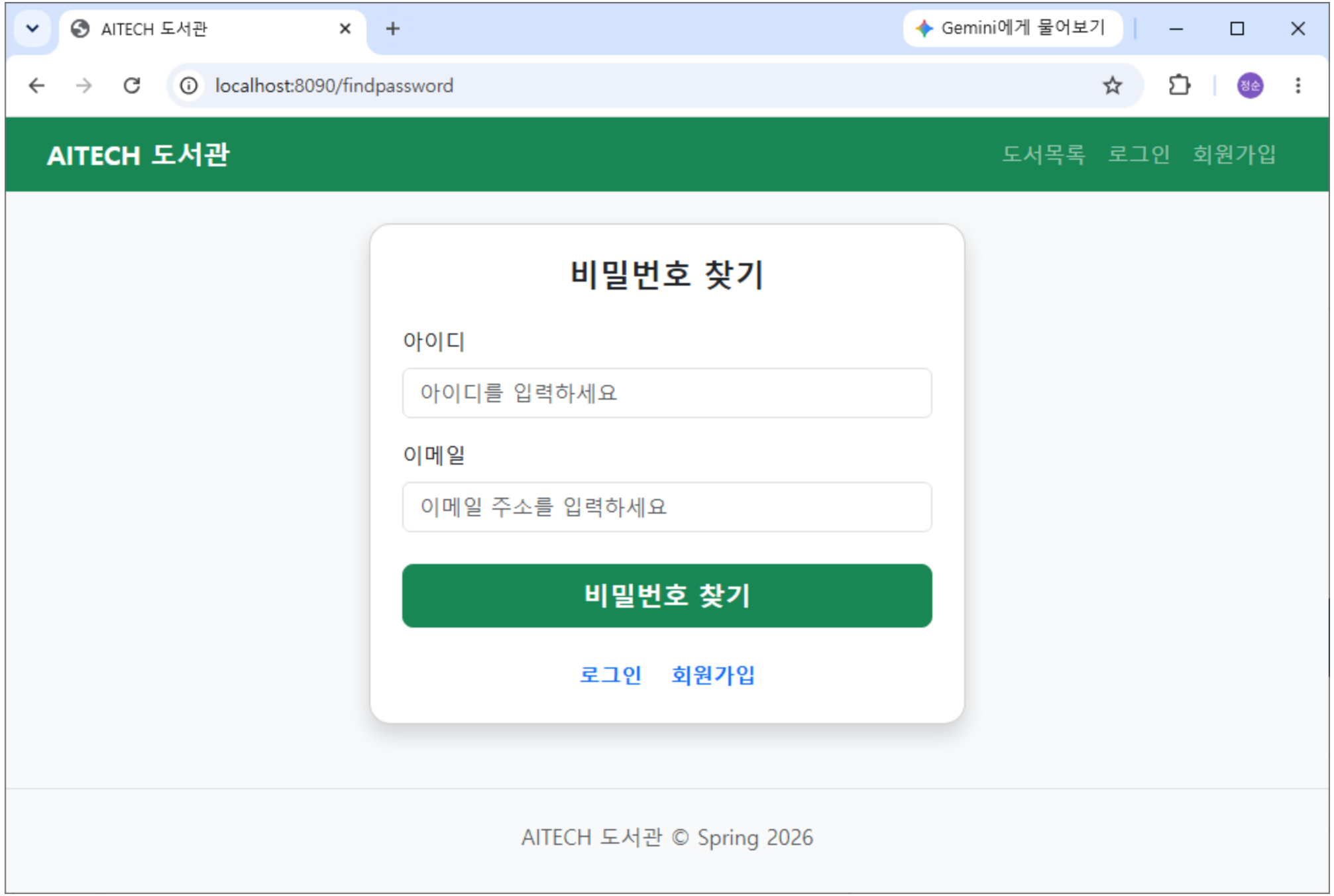
비밀번호 찾기

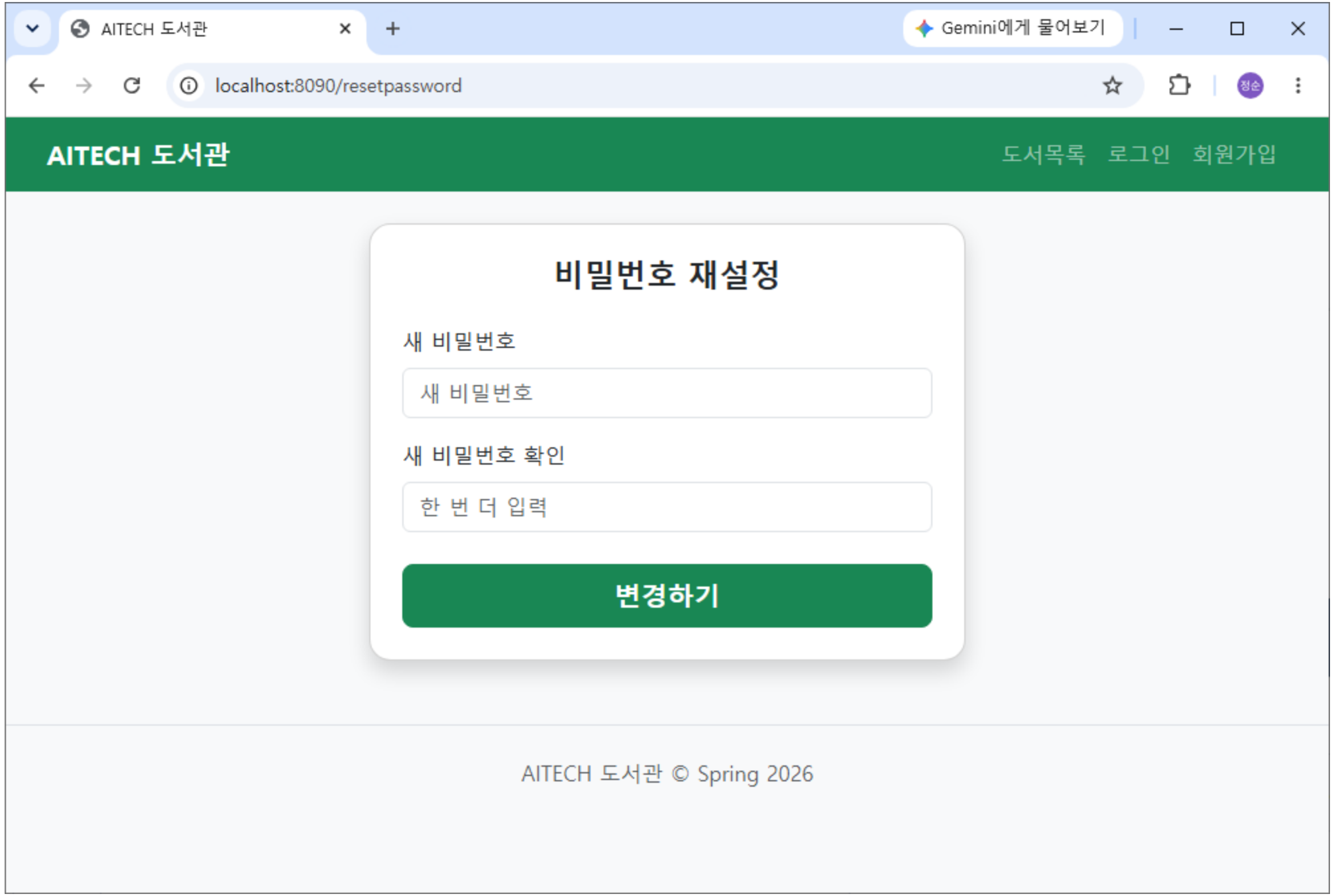
MemberController.java 수정

- resetpassword(), resetpasswordconfirm 추가

```
@RequestMapping("/resetpassword")
public String resetpassword(HttpSession session) {
    if (session.getAttribute("resetMember") == null) {
        return "redirect:/findpassword";
    }
    return "user/reset_password";
}

@RequestMapping("/resetpasswordconfirm") //회원정보로 초기화
public String resetpasswordconfirm(@RequestParam("m_pw") String m_pw, HttpSession session) {
    MemberVo resetMember = (MemberVo) session.getAttribute("resetMember");
    if (resetMember == null) {
        return "redirect:/findpassword";
    }
    memberDao.updatePassword(resetMember.getM_no(), passwordEncoder.encode(m_pw));
    session.removeAttribute("resetMember");
    return "redirect:/loginmember";
}
```


AITECH 도서관
Gemini에게 물어보기
localhost:8090/findpassword
AITECH 도서관
도서목록 로그인 회원가입
비밀번호 찾기
아이디
아이디를 입력하세요
이메일
이메일 주소를 입력하세요
비밀번호 찾기
로그인 회원가입
AITECH 도서관 © Spring 2026


AITECH 도서관
Gemini에게 물어보기
localhost:8090/resetpassword
AITECH 도서관
도서목록 로그인 회원가입
비밀번호 재설정
새 비밀번호
새 비밀번호
새 비밀번호 확인
한 번 더 입력
변경하기
AITECH 도서관 © Spring 2026

임시 비밀번호 메일 발송
GMAIL

임시 비밀번호 발송

요구 사항

- 아이디와 이메일이 DB의 회원과 동시에 일치하면 8자리 임시 비밀번호 생성 후 메일 발송 → 메일 발송 후 encode 된 임시 비밀번호 저장
- 사용자는 메일을 확인하여 발송된 임시 비밀번호로 로그인 후 정보 수정
- GMAIL 서비스 활용

임시 비밀번호 발송

pom.xml 의존성 추가

- MavenRepository → 메일 도구 spring mail

```
<!-- Source:
https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-mail -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

임시 비밀번호 발송

■ 구글 앱 비밀번호 발급

- 반드시 2단계 설정이 되어 있어야 함
- [Google 계정 관리] → [보안] → 2단계 인증 활성화(비활성화 경우 앱 비밀번호 메뉴가 안 보임)
- 보안 검색창에 " 앱 비밀번호 " 검색 → 앱 이름 아무거나(예: aitech) → [만들기]
- 노란 상자에 표시되는 16자리를 복사해 둔다 — 공백 없이 쓸 것



임시 비밀번호 발송

application.properties 수정

- Gmail 관련 SMTP 설정

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=본인 구글 계정
spring.mail.password=앱 비밀번호
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

임시 비밀번호 발송

MemberController.java 수정

- 메일 발송 도구 주입

```
@Autowired
private JavaMailSender mailSender;
```

임시 비밀번호 발송

MemberController.java 수정

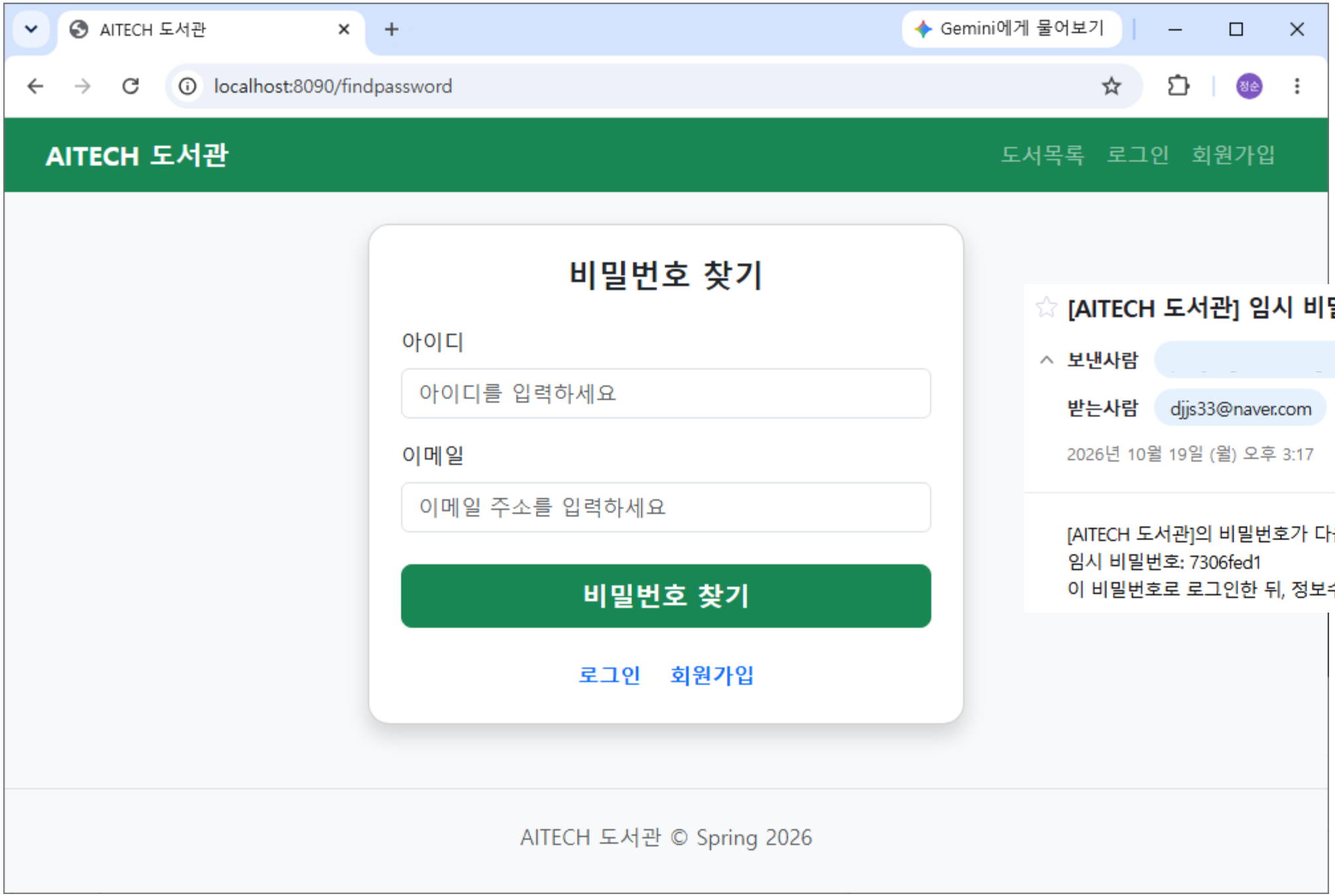
- findpasswordconfirm()을 주석 후 새로 생성

```
@RequestMapping("/findpasswordconfirm")
public String findpasswordconfirm(@RequestParam("m_id") String m_id, @RequestParam("m_mail") String m_mail) {
    MemberVo member = memberDao.getMemberByIdMail(m_id, m_mail);
    if (member == null) {
        return "redirect:/findpassword";
    }

    String tempPw = UUID.randomUUID().toString().substring(0, 8); // 임시 비밀번호 8자리

    SimpleMailMessage message = new SimpleMailMessage();
    message.setTo(member.getM_mail());
    message.setSubject("[AITECH 도서관] 임시 비밀번호 안내");
    message.setText("[AITECH 도서관]의 비밀번호가 다음과 같이 초기화 되었습니다.\n임시 비밀번호: " + tempPw
            + "\n이 비밀번호로 로그인한 뒤, 정보수정에서 새 비밀번호로 바꿔주세요.");
    try {
        mailSender.send(message);
    } catch (MailException e) {
        return "redirect:/findpassword"; // 발송 실패 → DB는 건드리지 않는다
    }

    memberDao.updatePassword(member.getM_no(), passwordEncoder.encode(tempPw));
    return "redirect:/loginmember";
}
```


AITECH 도서관
Gemini에게 물어보기
localhost:8090/findpassword
AITECH 도서관
도서목록 로그인 회원가입
비밀번호 찾기
아이디
아이디를 입력하세요
이메일
이메일 주소를 입력하세요
비밀번호 찾기
로그인 회원가입
AITECH 도서관 © Spring 2026


[AITECH 도서관] 임시 비밀번호 안내
보낸사람
받는사람 djjs33@naver.com
2026년 10월 19일 (월) 오후 3:17
[AITECH 도서관]의 비밀번호가 다음과 같이 초기화 되었습니다.
임시 비밀번호: 7306fed1
이 비밀번호로 로그인한 뒤, 정보수정에서 새 비밀번호로 바꿔주세요.

네이버 로그인
OAuth2 인가 코드

네이버 로그인

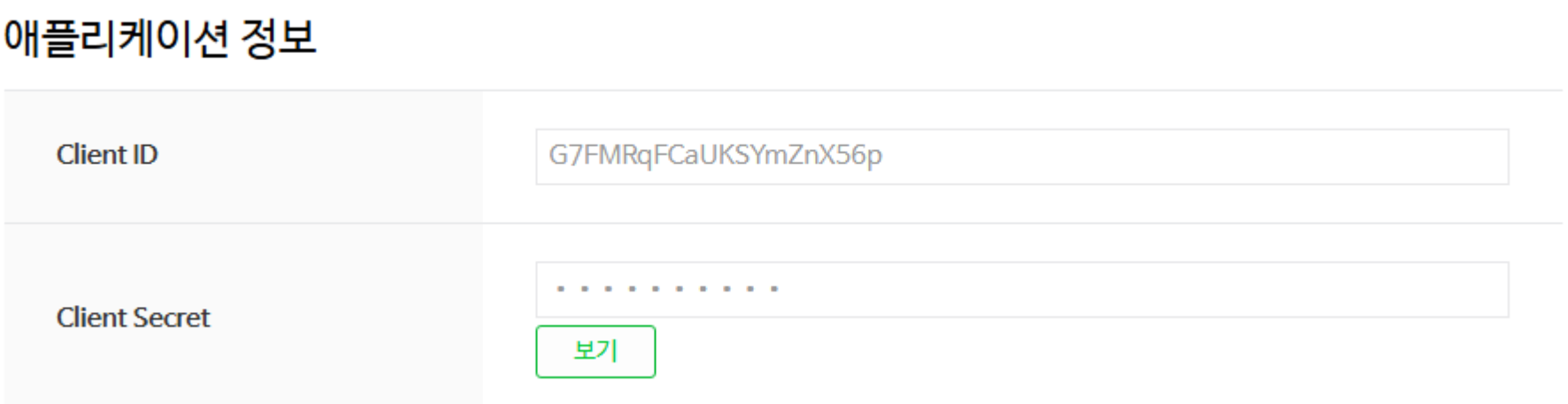
요구 사항

- 네이버 로그인 버튼을 클릭하면 네이버 이용 동의 후 프로필의 정보로 자동 가입되며, 해당 정보로 로그인이 가능하다.
- Id는 naver 계정의 고유번호로 사용한다.

네이버 로그인

Application 등록

- 네이버 개발자 센터 -> application 등록
- 서비스 URL http://localhost:8090
- Callback URL http://localhost:8090/naverlogincallback



네이버 로그인

application.properties 수정

- 부여 받은 Client ID와 Secret 등록

```
naver.client-id=부여받은 id
naver.client-secret=부여받은 번호
```

login_member.jsp 수정

- <a> 태그로 수정

```
<div class="d-grid gap-2">
    <button type="submit" class="btn btn-success">로그인</button>
    <a href="naverlogin" class="btn btn-secondary">N 네이버 로그인</a>
</div>
```

네이버 로그인

MemberController.java

- 필드 주입

```
@Value("${naver.client-id}")
private String naverClientId;

@Value("${naver.client-secret}")
private String naverClientSecret;
```

- 위조 방지를 만들어 세션에 등록 후 네이버 인가 URL 호출

```
@RequestMapping("/naverlogin")
public String naverlogin(HttpSession session) {
    String state = UUID.randomUUID().toString();
    session.setAttribute("naverState", state);

    String url = "https://nid.naver.com/oauth2.0/authorize?response_type=code" + "&client_id=" + naverClientId
            + "&redirect_uri="
            + URLEncoder.encode("http://localhost:8090/naverlogincallback", StandardCharsets.UTF_8) + "&state="
            + state;
    return "redirect:" + url;
}
```

네이버 로그인

MemberController.java

- 필드 주입

```
@Value("${naver.client-id}")
private String naverClientId;

@Value("${naver.client-secret}")
private String naverClientSecret;
```

- 위조 방지를 만들어 세션에 등록 후 네이버 인가 URL 호출, 출발 매핑

```
@RequestMapping("/naverlogin")
public String naverlogin(HttpSession session) {
    String state = UUID.randomUUID().toString();
    session.setAttribute("naverState", state);

    String url = "https://nid.naver.com/oauth2.0/authorize?response_type=code" + "&client_id=" + naverClientId
            + "&redirect_uri="
            + URLEncoder.encode("http://localhost:8090/naverlogincallback", StandardCharsets.UTF_8) + "&state="
            + state;
    return "redirect:" + url;
}
```

네이버 로그인

MemberController.java

- 도착 매핑

```
@RequestMapping("/naverlogincallback")
public String naverlogincallback(@RequestParam(value = "code", required = false) String code,
        @RequestParam(value = "state", required = false) String state, HttpSession session) {
    String savedState = (String) session.getAttribute("naverState");
    if (code == null || savedState == null || !savedState.equals(state)) {
        return "redirect:/loginmember";
    }
    session.removeAttribute("naverState");

    RestClient restClient = RestClient.create();

    URI tokenUri = UriComponentsBuilder.fromUriString("https://nid.naver.com/oauth2.0/token")
            .queryParam("grant_type", "authorization_code")
            .queryParam("client_id", naverClientId)
            .queryParam("client_secret", naverClientSecret)
            .queryParam("code", code).queryParam("state", state)
            .build().toUri();
    Map tokenRes = restClient.get().uri(tokenUri).retrieve().body(Map.class);
    String accessToken = (String) tokenRes.get("access_token");

    Map profileRes = restClient.get().uri("https://openapi.naver.com/v1/nid/me")
            .header("Authorization", "Bearer " + accessToken)
            .retrieve().body(Map.class);
```

네이버 로그인

aitech

전체 동의하기

네이버 개인정보 처리 동의

[필수] 개인정보 제 3자 제공 동의

이용자 식별자, 휴대전화번호, 이름, 이메일 주소, 성별

네이버는 회원가입/로그인 기능 제공자이며, aitech 서비스 제공자가 아닙니다. aitech 서비스 및 이용약관에 대한 의무와 책임은 aitech 에 있습니다. 동의 후에는 aitech의 이용약관 및 개인정보처리방침에 따라 정보가 관리됩니다.

취소

동의하기

Thank you!